



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №9

з дисципліни «Технології розроблення програмного забезпечення»

Тема: «13. Office communicator»

Виконав:

студент групи ІА-32
Шадлер Андрій

Перевірив:

вик. Мягкий М. Ю.

Тема роботи: Розробка клієнт–серверного застосунку з використанням ASP.NET Web API та Vue.js.

Мета роботи: Ознайомитися з принципами побудови клієнт–серверної архітектури, розробити серверну частину у вигляді веб-API на базі ASP.NET та клієнтську частину у вигляді односторінкового застосунку (SPA) з використанням фреймворку Vue.js, а також забезпечити їх взаємодію через HTTP-запити.

Зміст

Теоретичні відомості.....	2
Хід виконання:.....	Помилка! Закладку не визначено.
Завдання.....	Помилка! Закладку не визначено.
Діаграма розгортання системи	Помилка! Закладку не визначено.
Діаграма компонентів	Помилка! Закладку не визначено.
Діаграма послідовностей	Помилка! Закладку не визначено.
Вихідний код	6
Контрольні запитання.....	23
Висновок	25

Теоретичні відомості

Клієнт–серверна архітектура

Клієнт–серверна архітектура – це модель взаємодії, у якій:

- **сервер** відповідає за обробку запитів, бізнес-логіку та доступ до даних;
- **клієнт** відповідає за інтерфейс користувача та ініціацію запитів до сервера.

Взаємодія між клієнтом і сервером зазвичай відбувається за допомогою протоколу **HTTP** та форматів обміну даними, таких як **JSON**.

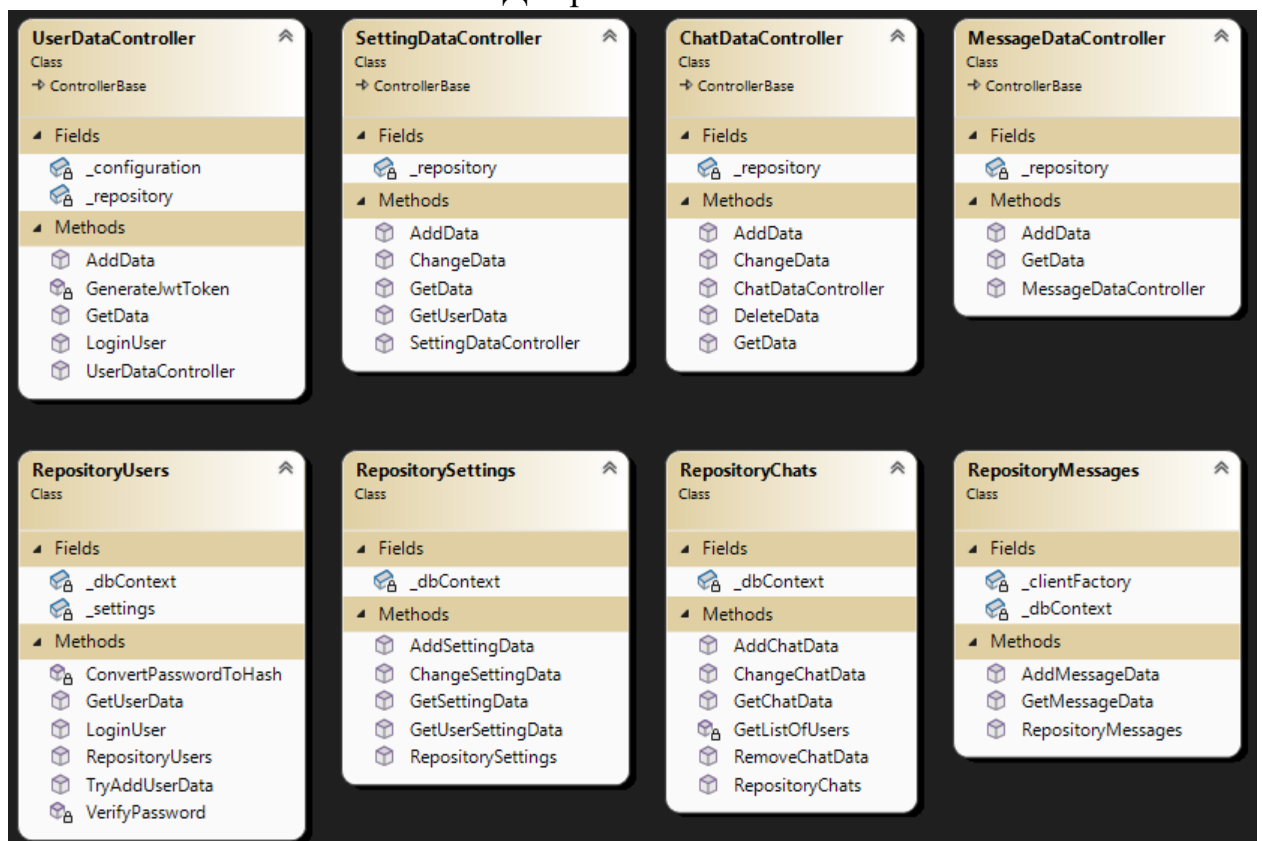
Хід виконання:

Завдання:

- Ознайомитись з короткими теоретичними відомостями.
- Реалізувати частину функціоналу робочої програми у вигляді класів та їхньої взаємодії для досягнення конкретних функціональних можливостей.

- Реалізувати функціонал для роботи в розподіленому оточенні відповідно до обраної теми.
- Реалізувати взаємодію розподілених частин:
 - Для клієнт-серверних варіантів: реалізація клієнтської і серверної частини додатків, а також загальної частини (middleware); зв'язок клієнтської і серверної частин за допомогою WCF, TcpClient, .NET Remoting на розсуд виконавця.
- Підготувати звіт щодо виконання лабораторної роботи. Поданий звіт повинен містити: діаграму класів, яка представляє спроектовану архітектуру. Навести фрагменти програмного коду, які є суттєвими для відображення реалізованої архітектури.

Діаграма класів:



Діаграма класів на беку. Методи фронту реалізовані в 1 файлі – у `app.js`

Опис реалізації:

Опис архітектури застосунку

Розроблений застосунок складається з двох незалежних частин:

1. Серверна частина – ASP.NET Web API.
2. Клієнтська частина – фронтенд на Vue.js.

Ці частини взаємодіють між собою через REST-ендпоінти.

Серверна частина (ASP.NET Web API)

Сервер реалізовано у вигляді веб-застосунку на платформі ASP.NET, який надає REST-API для клієнтської частини.

Основні компоненти серверної частини:

- Контролери – обробляють HTTP-запити (GET, POST, PUT, DELETE);
- Ендпоінти – точки доступу до ресурсів застосунку;
- Моделі (DTO) – використовуються для передачі даних між клієнтом і сервером;
- Репозиторії / сервіси – містять бізнес-логіку та логіку доступу до даних;
- Аутентифікація та авторизація (за наявності) – забезпечують захист API.

Сервер повертає відповіді у форматі JSON, що є зручним для обробки на стороні клієнта.

Клієнтська частина (Vue.js)

Клієнтська частина реалізована як односторінковий застосунок (SPA) з використанням фреймворку Vue.js.

Основні компоненти клієнтської частини:

- Vue-компоненти – відповідають за відображення інтерфейсу користувача;
- Сервіси для HTTP-запитів (наприклад, fetch або axios);
- Стан застосунку (data, computed, methods);
- Маршрутизація (за потреби) для переходу між сторінками без перезавантаження.

Клієнт ініціює HTTP-запити до серверних ендпоінтів та відображає отримані дані користувачу.

Взаємодія між клієнтом і сервером

Взаємодія відбувається за наступною схемою:

1. Користувач виконує дію в інтерфейсі Vue-застосунку.
2. Клієнт формує HTTP-запит до відповідного ендпоінту сервера.
3. Сервер обробляє запит у контролері та повертає відповідь.
4. Клієнт отримує відповідь у форматі JSON та оновлює інтерфейс.

Такий підхід забезпечує чітке розділення відповідальностей між клієнтом і сервером.

Переваги обраної архітектури

- чітке розділення бізнес-логіки та інтерфейсу користувача;
- можливість незалежної розробки клієнта і сервера;
- зручне масштабування та підтримка застосунку;
- повторне використання серверного API іншими клієнтами (мобільними або десктопними).

Недоліки та обмеження

- залежність від мережевого з'єднання;
- необхідність обробки помилок та затримок HTTP-запитів;
- ускладнення архітектури порівняно з монолітними застосунками.

Вихідний код

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.Tokens;
using Skype.Constants;
using Skype.Data;
using Skype.Repositories;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;

namespace Skype.Controllers
{
    [Route("[controller]")]
    [ApiController]
    public class UserDataController : ControllerBase
    {
        RepositoryUsers _repository;

        IConfiguration _configuration;

        public UserDataController(RepositoryUsers repository, IConfiguration
configuration)
        {
            _repository = repository;
            _configuration = configuration;
        }

        private string GenerateJwtToken(string id, string role, string username, string
name)
```

```

{
    var claims = new[]
    {
        new Claim("id", $"{id}"),
        new Claim(ClaimTypes.Role, $"{role}"),
        new Claim("username", $"{username}"),
        new Claim("name", $"{name}")
    };

    var secretKey = new
    SymmetricSecurityKey(Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]));

    var loginCredentials = new SigningCredentials(secretKey,
    SecurityAlgorithms.HmacSha256);

    var tokenOptions = new JwtSecurityToken
    (
        issuer: _configuration["Jwt:Issuer"],
        audience: _configuration["Jwt:Audience"],
        claims,
        expires: DateTime.UtcNow.AddMinutes(60),
        signingCredentials: loginCredentials
    );

    var tokenString = new
    JwtSecurityTokenHandler().WriteToken(tokenOptions);

    return tokenString;
}

[HttpGet]
[Authorize(Roles = "Support")]
public async Task<List<User>> GetData()

```

```
{  
    return await _repository.GetUserData();  
}
```

[HttpPost]

[AllowAnonymous]

```
public async Task<IActionResult> AddData( [FromForm] string name,  
                                           [FromForm] string username,  
                                           [FromForm] string phone,  
                                           [FromForm] string password,  
                                           [FromForm] UserRole role)  
{  
    if (!await _repository.TryAddUserData(name, username, phone, password,  
role))  
    {  
        return Conflict("User already exist");  
    }  
  
    return Ok();  
}
```

[HttpPost("authenticate")]

[AllowAnonymous]

```
public async Task<IActionResult> LoginUser( [FromForm] string username,  
                                           [FromForm] string password)  
{  
    var user = await _repository.LoginUser(username, password);  
  
    if (user == null)
```



```

    {
        return Unauthorized();
    }

    var id = user.Id.ToString();
    var role = user.Role.ToString();
    var name = user.Name.ToString();

    var usernameNotLower = user.Username.ToString();

    var token = GenerateJwtToken(id, role, usernameNotLower, name);

    return Ok( new { token } );
}
}
}

```

```

const { createApp } = Vue;
const API_URL = 'https://localhost:7272/';

```

```

createApp({
  data() {
    return {
      isLogin: true,
      isAuthenticated: false,
      showSettings: false,
      showCreateChat: false,
      showChatSettings: false,
      showCallMenu: false,

```

messageRefreshInterval: null,

user: {

name: "",

username: "",

phone: "",

password: "",

id: "",

role: "",

},

settings: {

bgColor: '#ffffff',

fontSize: 16

},

chats: {

chatsList: [],

selectedChat: null

},

newChat: {

name: "",

description: "",

usernames: ""

},

```

editChat: {
  name: "",
  description: "",
  usernames: ""
},

messages: {
  messagesList: [],
  newMessage: ""
},
}
},
mounted(){
  this.startAutoRefresh();
},
beforeUnmount() {
  this.stopAutoRefresh();
},
methods: {
  toggleMode() {
    this.isLogin = !this.isLogin;
  },
  toggleCallMenu() {
    this.showCallMenu = !this.showCallMenu;
  },
  startAutoRefresh() {
    this.messageRefreshInterval = setInterval(() => {
      if (this.chats.selectedChat) {
        this.getMessageData(this.chats.selectedChat.Id);

```

```

    }
    }, 30000); // Refresh every 30 seconds
  },
  stopAutoRefresh() {
    if (this.messageRefreshInterval) {
      clearInterval(this.messageRefreshInterval);
      this.messageRefreshInterval = null;
    }
  },
  async parseJWT(token) {
    if (!token) return null;

    const payloadBase64Url = token.split('.')[1];
    const base64 = payloadBase64Url
      .replace(/-/g, '+')
      .replace(/_/g, '/')
      .padEnd(payloadBase64Url.length + (4 - payloadBase64Url.length % 4)
% 4, '=');
    const binary = atob(base64);
    const payloadJson = decodeURIComponent(
      binary
        .split("")
        .map(c => '%' + ('00' + c.charCodeAt(0).toString(16)).slice(-2))
        .join("")
    );
    const payload = JSON.parse(payloadJson);

    let role = "http://schemas.microsoft.com/ws/2008/06/identity/claims/role";
    // Path of Claims.Roles

```

```

    this.user.role = payload[role];
    this.user.id = payload.id;
    this.user.username = payload.username;
    this.user.name = payload.name;

},
async submitLoginRegisterForm() {
    switch (this.isLogin) {
        case true:
            await this.login();
            break;
        case false:
            await this.register();
            break;
    }
},
async loadSettings() {
    await axios.get(API_URL + `SettingData/${this.user.id}`, {
        headers: {
            'Authorization': `Bearer ${localStorage.getItem('token')}`
        }
    }).then(response => {
        this.settings.bgColor = response.data[0].BgColour;
        this.settings.fontSize = response.data[0].FontSize;
        this.applySettingsToCSS();
    });
},
async applySettings() {

```

```

const formData = new FormData();
formData.append("id", this.user.id);
formData.append("bgColour", this.settings.bgColor);
formData.append("fontSize", this.settings.fontSize);

await axios.put(API_URL + "SettingData", formData, {
  headers: {
    'Authorization': `Bearer ${localStorage.getItem('token')}`
  }
}).then(() => {
  this.applySettingsToCSS();
});
},
applySettingsToCSS() {
  document.documentElement.style.setProperty('--bg-color',
this.settings.bgColor);

  document.documentElement.style.setProperty('--font-size',
this.settings.fontSize + 'px');
},
async login() {
  if (!this.user.username || !this.user.password) {
    alert("Заповніть всі поля!");
    return;
  }

  const formData = new FormData();
  formData.append("username", this.user.username);
  formData.append("password", this.user.password);

```

```

await axios.post(API_URL + "UserData/authenticate", formData)
  .then(response => {
    this.user.username = "";
    this.user.password = "";
    localStorage.setItem('token', response.data.token);
    this.parseJWT(localStorage.getItem('token'));
    this.getChatData();
    this.loadSettings();
    this.isAuthenticated = true;
  })
  .catch(error => {
    alert("Невірний логін або пароль!");
    console.error(error);
  });
},
async register() {
  if (!this.user.name || !this.user.username || !this.user.phone ||
!this.user.password) {
    alert("Заповніть всі поля!");
    return;
  }

```

```

const formData = new FormData();
formData.append("name", this.user.name);
formData.append("username", this.user.username);
formData.append("phone", this.user.phone);
formData.append("password", this.user.password);
formData.append("role", 1); // Default role: User

```

```

    await axios.post(API_URL + "UserData", formData)
      .then(response => {
        this.toggleMode();
        alert("Реєстрація успішна! Тепер ви можете увійти.");
      })
      .catch(error => {
        console.error(error);
        alert("Помилка реєстрації. Спробуйте ще раз.");
      });
  },
  async getChatData() {
    await axios.get(API_URL + `ChatData/${this.user.id}`, {
      headers: {
        'Authorization': `Bearer ${localStorage.getItem('token')}`
      }
    })
      .then(response => {
        this.chats.chatsList = response.data;
      });
  },
  async createChat() {
    if (!this.newChat.name || !this.newChat.description ||
    !this.newChat.usernames) {
      alert("Заповніть всі поля!");
      return;
    }

    const usernamesArray = this.newChat.usernames
      .split(',')

```



```

        .map(name => name.trim().toLowerCase())
        .filter(name => name.length > 0);

const formData = new FormData();
formData.append("name", this.newChat.name);
formData.append("description", this.newChat.description);

usernamesArray.forEach(username => {
    formData.append("usernames", username);
});

await axios.post(API_URL + "ChatData", formData, {
    headers: {
        'Authorization': `Bearer ${localStorage.getItem('token')}`
    }
})
    .then(() => {
        this.newChat.name = "";
        this.newChat.description = "";
        this.newChat.usernames = "";
        this.showCreateChat = false;
        this.getChatData();
    });
},

async editSelectedChat() {
    if (!this.editChat.name || !this.editChat.description ||
        !this.editChat.usernames) {
        alert("Заповніть всі поля!");
    }
}

```

```

    return;
  }

  const usernamesArray = this.editChat.usernames
    .split(',')
    .map(name => name.trim().toLowerCase())
    .filter(name => name.length > 0);

  const formData = new FormData();
  formData.append("id", this.chats.selectedChat.Id);
  formData.append("name", this.editChat.name);
  formData.append("description", this.editChat.description);

  usernamesArray.forEach(username => {
    formData.append("usernames", username);
  });

  await axios.put(API_URL + "ChatData", formData, {
    headers: {
      'Authorization': `Bearer ${localStorage.getItem('token')}`
    }
  })
  .then(() => {
    this.chats.selectedChat.Name = this.editChat.name;
    this.chats.selectedChat.Description = this.editChat.description;
    this.editChat.name = "";
    this.editChat.description = "";
    this.editChat.usernames = "";
    this.showChatSettings = false;
  });

```

```

        this.getChatData();
    });
},
async deleteChat() {
    if (!confirm("Ви впевнені, що хочете видалити цей чат?")) {
        return;
    }

    await axios.delete(API_URL + `ChatData/${this.chats.selectedChat.Id}`, {
        headers: {
            'Authorization': `Bearer ${localStorage.getItem('token')}`
        }
    });

    this.chats.selectedChat = null;
    this.messages.messagesList = [];
    this.getChatData();
},
async getMessageData(chatId) {
    await axios.get(API_URL + `MessageData/${chatId}`, {
        headers: {
            'Authorization': `Bearer ${localStorage.getItem('token')}`
        }
    })
    .then(response => {
        this.messages.messagesList = response.data;
    });
},
async sendMessage() {

```

```

    if (!this.messages.newMessage) {
        return;
    }

    const formData = new FormData();
    formData.append("chatId", this.chats.selectedChat.Id);
    formData.append("ownerId", this.user.id);
    formData.append("text", this.messages.newMessage);

    await axios.post(API_URL + "MessageData", formData, {
        headers: {
            'Authorization': `Bearer ${localStorage.getItem('token')}`
        }
    });

    this.messages.newMessage = "";
    await this.getMessageData(this.chats.selectedChat.Id);

    this.$nextTick(() => {
        this.ScrollToBottom();
    });
},
async selectChat(chat) {
    this.chats.selectedChat = chat;
    await this.getMessageData(chat.Id);

    this.$nextTick(() => {
        this.ScrollToBottom();
    });
}

```

```
},  
openChatSettings() {  
    this.editChat.name = this.chats.selectedChat.Name;  
    this.editChat.description = this.chats.selectedChat.Description;  
  
    this.editChat.usernames = this.chats.selectedChat.Usernames.join(', ');  
  
    this.showChatSettings = true;  
},  
formatTime(timestamp) {  
    if (!timestamp.endsWith('Z')) {  
        timestamp += 'Z';  
    }  
  
    const date = new Date(timestamp);  
    return date.toLocaleTimeString([], { hour: "2-digit", minute: "2-digit" });  
},  
ScrollToBottom() {  
    const el = this.$refs.msgContainer;  
    el.scrollTop = el.scrollHeight;  
},  
async startMeetCall() {  
    const link = "https://meet.google.com/new"  
  
    this.messages.newMessage = "Долучайтесь до Meet дзвінка: ";  
  
    this.toggleCallMenu();  
  
    window.open(link, '_blank');
```

```

    },
    async startJitSiCall() {
        const room = "chat_" + this.chats.selectedChat.Id + "_" +
this.chats.selectedChat.Name;
        const link = `https://meet.jit.si/${room}`;

        this.messages.newMessage = "Долучайтесь до Jitsy дзвінка: " + link;
        await this.sendMessage();
        this.toggleCallMenu();
        window.open(link, '_blank');
    }
},
/* watch: {
    'chats.selectedChat': function () {
        this.$nextTick(() => {
            this.ScrollToBottom();
        });
    },
    'messages.messagesList': function () {
        this.$nextTick(() => {
            this.ScrollToBottom();
        });
    }
} */
}).mount('#app');
```

Контрольні запитання

1. Що таке клієнт-серверна архітектура?

Клієнт-серверна архітектура – це модель взаємодії, у якій клієнт ініціює запити, а сервер обробляє їх, виконує бізнес-логіку та повертає результати. Клієнт і сервер мають чітко розмежовані ролі.

2. Розкажіть про сервіс-орієнтовану архітектуру (SOA)

Service-Oriented Architecture (SOA) – це архітектурний підхід, у якому функціональність системи реалізується у вигляді незалежних сервісів, що взаємодіють між собою через стандартизовані інтерфейси.

3. Якими принципами керується SOA?

Основні принципи SOA:

- слабка зв'язаність сервісів;
- чітко визначені контракти;
- повторне використання сервісів;
- автономність сервісів;
- незалежність реалізації від споживачів;
- можливість композиції сервісів.

4. Як між собою взаємодіють сервіси в SOA

Сервіси взаємодіють шляхом обміну повідомленнями через мережу, зазвичай з використанням стандартних протоколів (HTTP, SOAP) і форматів даних (XML, JSON).

5. Як розробники дізнаються про існуючі сервіси і як робити до них запити?

Розробники дізнаються про сервіси через:

- документацію (OpenAPI, WSDL);
- реєстри сервісів;

- сервісні каталоги.

Запити виконуються через стандартизовані API з чітко описаними контрактами.

6. Переваги та недоліки клієнт-серверної моделі

Переваги:

- централізоване управління даними;
- підвищена безпека;
- простота підтримки та оновлення;
- чіткий розподіл відповідальностей.

Недоліки:

- залежність від сервера;
- можливі проблеми масштабування;
- затримки через мережу;
- єдина точка відмови.

7. Переваги та недоліки однорангової (peer-to-peer) моделі

Переваги:

- відсутність центрального сервера;
- висока масштабованість;
- підвищена відмовостійкість.

Недоліки:

- складність керування;
- проблеми безпеки;
- складна синхронізація даних;
- залежність від надійності кожного вузла.

8. Що таке мікросервісна архітектура?

Мікросервісна архітектура – це підхід, у якому система складається з набору невеликих, автономних сервісів, кожен з яких реалізує окрему бізнес-функцію та може розгортатися незалежно.

9. Які протоколи використовуються в мікросервісній архітектурі?

Найпоширеніші протоколи:

- HTTP/REST;
- HTTPS;
- gRPC;
- AMQP (RabbitMQ);
- Kafka (подієва взаємодія).

10. Чи є SOA шар бізнес-логіки між контролерами та доступом до даних?

Ні, формально – ні.

Такий підхід є **шаровою архітектурою**, а не SOA, оскільки:

- сервіси не є автономними;
- вони не мають мережових контрактів;
- не можуть використовуватися незалежно іншими системами.

Проте концептуально він використовує деякі ідеї SOA (розділення відповідальностей).

Висновок

У ході виконання лабораторної роботи було розроблено клієнт–серверний застосунок, у якому серверна частина реалізована за допомогою ASP.NET Web API, а клієнтська – з використанням Vue.js. Реалізована архітектура забезпечує гнучку, масштабовану та зручну для підтримки систему з чітким розділенням відповідальностей між компонентами.